

摘要：

Dean表示，AI已达初级工程师水平并可连续运行数周。随着智能体能力超预期，核心瓶颈正转向推理硬件效率。低延迟专用硬件、上下文工程与专用领域模型是小团队差异化的关键，未来AI与科学研究将加速闭环演进。

AI智能体正从“工具”演变为能够连续运行数周、独立完成复杂任务的系统，而制约其大规模普及的核心瓶颈，正在从模型能力转向推理硬件效率。Google首席科学家、深度学习先驱Jeff Dean在Y Combinator的访谈中，对AI现状与未来走向作出了一系列研判，并点明了他眼中下一个技术突破口所在。

Dean表示，他去年对“AI已达初级工程师水平”的预测基本得到了验证，且模型在完成更复杂任务方面的进展速度超出了他的预期。他尤其强调，AI智能体在编程之外的领域同样开始展现能力，这将是未来的重要趋势。

在预判下一阶段突破口时，Dean将目光指向推理硬件。他提出，若推理延迟能降低50倍，将从根本上改变AI系统的产品边界，催生出目前难以实现的全新应用形态。与此同时，他对创业者发出明确信号：上下文工程、专用领域模型，以及能够整合推理时计算的多智能体系统，将是小团队实现差异化的主要窗口。



AI智能体能力已超预期，可连续运行数周

Dean在对话中证实，其2025年5月作出的“AI达到初级工程师水平”判断基本准确，并指出实际进展在某些维度上超出了他的预期。“模型完成更复杂任务的能力增长速度比我预想的要快，”他说，“而且在编程之外的领域，这些基于智能体的系统也开始崭露头角。”

他认为，目前市场对AI智能体能力存在一个被普遍低估的认知盲点：**基于智能体的系统已经可以不止运行一两个小时，在某些问题领域、配合足够强大的模型，可以连续运行数天甚至数周，完成极为复杂的任务。**“有些人已经开始隐约察觉到这一点，但我认为还没有被普遍接受，这将是一件影响很大的事。”

在实际应用层面，Dean举例称，他和同事Sanjay近期在做底层库性能优化时，通过为智能体编写“技能”（skill），教会模型按顺序完成“测量基准—改进代码—测量性能提升—迭代”的完整流程，实现了相当高效的自动化循环。他还提到，目前的模型在将软件从一种编程语言翻译成另一种语言方面已相当高效，因为原始代码本身构成了一份极其清晰的规格说明。

推理硬件：下一个关键战场

Dean将推理硬件效率定义为AI系统下一步大规模普及的核心约束，并类比了他2001年主导的一项历史性决策——将Google搜索索引从硬盘迁移至内存，以此说明硬件架构的跃迁对产品能力的决定性影响。

"大家会看到越来越多高性能、低能耗的推理硬件系统，"他表示，"**推理是让基于智能体的系统能够普及给更多人的关键，延迟非常重要，而硬件的专用化正是提升能效、降低延迟的关键途径，比GPU或TPU这类通用计算设备更有效。**"

Dean进一步指出，当前AI系统设计中存在一个被多数创业者忽视的底层约束：将数据从加速器的HBM取到处理器进行计算，所消耗的能量是实际执行一次乘法运算的上千倍。这一差距深刻影响了批处理的必要性、系统架构设计乃至产品的延迟表现。"如果没有这上千倍的差距，就不需要做批处理；但正因为有，你必须一次性处理很多样本或很多token，才能把这次数据搬运的开销摊薄。"

在推理硬件的优化方向上，他着重提到两点：尽量减少数据搬运，以及尝试极低精度运算，将所需精度直接内建到硬件中，而非支持冗余的精度选项。

上下文工程崛起，小团队迎来差异化窗口

Dean指出，**AI进步的驱动力正在发生结构性转变——从"更大的模型、更多的数据"**，逐步转向模型之外的系统工程能力，包括检索、工具调用、记忆管理和智能体编排，即业界所称的"上下文工程" (context engineering)。

"模型只是你要构建的整体系统中的一部分，"他说，"这涉及模型如何使用各种工具，如何检索相关信息，可能还要保留过去解决其他问题时检索到的信息，并把这些信息放进模型的上下文中。"

他将上下文工程视为小团队的重要机会入口，原因在于门槛结构的改变："以前训练一个模型需要海量的资源、GPU和数据，但做上下文工程，你只需要一个类似Gemini的API接口，然后自己搭建检索系统、工具调用之类的东西就行了。"

对于智能体在执行超过三四十步后容易"脱轨"的普遍问题，Dean给出了两类应对策略：一是为模型提供技能和提示，使其尽量停留在熟悉的操作路径上；二是采用多智能体架构，通过推理时计算 (inference-time compute) 搜索可行解题路径，由另一个模型或智能体对多条路径进行评估和筛选，保留有效方案，剔除偏离轨道的路径。

对创业者的判断框架：找模型成功率接近零的问题

在与创业者直接相关的议题上，Dean提供了一套较为具体的机会识别框架。

他指出，通用模型正在越来越多的领域持续变强，这意味着创业者必须审慎评估自己选择的方向是否具备足够的持续性："你正在做的事情是不是够'持久'——未来六个月或十二个月，前沿模型会不会就能做到这件事了？"

在识别有效切入点时，他给出了较为清晰的信号：**应当寻找当前通用模型成功率接近0%或1%的问题，而非20%**。后者往往意味着相关能力正处于萌芽阶段，随训练数据增加或模型规模扩大，通用模型可能很快追平。

他认为两类场景具有较强的结构性优势：其一，产品能够接触到通用模型无法获取的特殊数据，例如用户个人信息；其二，针对极其困难的特定问题，通过训练专用小模型，可以以较低成本获得高精度解法。他以AlphaFold为例——这是一个专用于蛋白质折叠的模型，并非通用系统，在特定领域取得了显著成效。他认为材料科学、芯片设计等领域同样存在类似机会。

自动化科学方法：AI加速AI自身演进

Dean对2027年的大胆预测指向一个更深层的演进方向：**机器学习系统本身的自动化程度将大幅提高，形成“用AI优化AI”的闭环。**

他描述的路径是：将问题拆解成子问题，在紧密的自动化实验循环中运行，再将结果整合，从而持续迭代出改进后的系统。“这不只适用于机器学习，也适用于其他科学和工程领域——基本上任何有可衡量目标的领域，现在都能取得很大进展。”

Dean还提到，加快这一循环的关键之一，是构建速度更快的验证模型。他以量子化学领域的案例说明：原本需要运行一整夜的密度泛函理论模拟，在用神经网络近似后速度提升了30万倍，使原本需要六个月才能完成的千万种分子构型筛选，在午饭时间内即可完成，从根本上改变了科学研究的节奏。

他同时提出了一个颇具挑战性的思想实验：过去60年，半导体行业默认晶体管必须极度可靠；但若在更高系统层面引入冗余容错机制，是否可以使用每天出错20次的晶体管来构建系统？这类对基础假设的重新审视，正是他认为能够催生下一代突破性系统的思维方式。

未来稀缺能力：判断力，而非执行力

面对“当智能体完成所有代码编写后，什么才是真正稀缺的”这一问题，Dean给出了明确的答案：**判断力（taste）——即知道该让智能体去解决什么问题。**

“模型未必擅长做这种判断，”他说，“未来会是人来引导大量的AI辅助计算，从而更快地实现伟大的成果——但那个‘你想要模型做什么’的核心，才是你应该专注的关键。”

他提出了一种量化训练判断力的方式：定期写下未来12个月内可能重要的事情清单，一年后回头核验哪些真的变得重要，哪些已被他人完成，哪些尚属空白。此外，他还强调了“第一性原理”思考的价值——不纠结于问题今天是怎么解决的，而是眯着眼睛审视问题的本质，寻找能带来一到两个数量级提升的截然不同的解决思路。



以下为访谈实录：

主持人： Jeff，欢迎你，非常感谢你能来。尤其是我刚感冒了，还是要谢谢你能来。

Jeff Dean： 我这几天嗓子哑了，平时说话不是这个样子，但我们尽量。

主持人： 你主导开发了MapReduce、Bigtable、TensorFlow、TPU、Gemini，光是这些成果就能聊上一两个小时。但我最欣赏的一点是，你现在依然愿意公开做出大胆的预测。去年，也就是2025年5月的AI Ascent大会上，你说AI已经达到了初级工程师的水平。这已经是一年前的事了，现在这个预测兑现得怎么样？

Jeff Dean： 我感觉模型在基于智能体的长程编程任务上进步很快，现在它们已经相当能干了。具体要看你怎么定义“初级工程师”，但我觉得这个预测基本说中了。

主持人： 这个预测里，你低估了什么？

Jeff Dean： 我觉得模型完成更复杂任务的能力增长速度比我预想的要快。而且我发现，在编程之外的领域，这些基于智能体的系统也开始崭露头角，我认为这会是未来的一个重要趋势。

主持人： 那再给我们一个大胆的预测吧，你觉得2027版的预测会是什么？

Jeff Dean： 我认为机器学习系统本身的自动化程度会大幅提高——让机器学习系统通过运行大量实验来自我提升能力，把问题拆解成子问题，在一个紧密的自动化实验循环里运行这些子问题，再把结果整合起来，从而得到一个改进后的系统。这种完全自动化的问题分解和自动化实验，我觉得会非常令人兴奋。而且这不只适用于机器学习，也适用于其他科学和工程领域——基本上任何有可衡量目标的领域，现在都能取得很大进展。

主持人： 我们回顾一下历史。早在2001年，Google搜索还跑在硬盘上，你和Sanjay算了一笔账，发现总有一天整个搜索索引能装进所有服务器的内存里。你们意识到这一点后，几天之内就把搜索从跑在硬盘上换成了跑在内存里，这也是Google搜索变快的关键。放到2026年的今天，“能装进内存”这样的时刻是什么？在座的每个人现在应该在思考、设计什么？

Jeff Dean： 情况不太一样了，但我认为大家会看到越来越多高性能、低能耗的推理硬件系统。因为大家现在都意识到，推理是让这些基于智能体的系统能够普及给更多人的关键，延迟非常重要，而硬件的专用化正是提升能效、降低延迟的关键途径，比GPU或TPU这类通用计算设备更有效。毕竟现在每个人都已经习惯了等待模型的响应。

主持人： 所以你是说，如果我们不用再等待了会怎样？

Jeff Dean： 对，设想一下，如果延迟能降低50倍，你能用它做什么。

主持人： 有意思的想法。现在，在座这6000人当中，有什么关于AI的普遍假设其实已经不成立了？

Jeff Dean： 我觉得可能有一点大家没有充分意识到，那就是基于智能体的系统其实已经可以不止运行一两个小时，在某些问题领域，配合足够强大的模型，它们可以连续运行数天甚至数周，去完成非常复杂的任务。有些人已经开始隐约察觉到这一点，但我认为还没有被普遍接受，这将会是一件影响很大的事。

主持人： 具体来说，有没有哪个任务是你让智能体跑了好几周的？你让它去解决什么问题了？

Jeff Dean： 比如说，你可以让智能体去用不同的编程语言，从头实现全新版本的软件，可能是安全性更好，或者性能更好的版本，然后让它们真正认真地去完成这件事。



主持人： 很酷。现在说说你特别擅长的一件事——用餐巾纸做估算。听起来有点好笑，但关于你的一个故事是：2013年Google的语音识别刚开始起作用时，你在餐巾纸上算了一笔账——如果每个Google用户每天用手机做三分种的语音识别，你发现这需要把整个Google服务器机队翻一倍，成本会非常高昂。而你没有这么做，你们自己造了一款定制芯片，这就是TPU的起源。

Jeff Dean： 对。当时我们开始在基于深度学习的语音系统上看到非常好的质量结果，但相比旧的语音系统，这些模型的计算开销要大得多，不过错误率也降低了很多，相当于几个月内就实现了语音识别20年的进步——只是稍微调了调模型、扩大了规模、用了更好的数据。于是我们开始担心，如果语音识别效果变得好很多，人们会更多地使用它，比如用语音口述邮件或者跟手机对话。我们意识到需要比当时用CPU运行更好的解决方案，于是就有了TPU——它专门针对低精度密集线性代数运算做了优化，而这正是当今几乎所有现代机器学习算法的核心。如果你为低精度密集线性代数专门造一块芯片，别的什么都不做，它对机器学习推理会非常有用，尽管它跑不了Chrome或Word之类的程序。几年后，这套系统做出的芯片，能效比当时的CPU和GPU高出30到80倍，延迟也低了20到30倍——这也是今天TPU能成为基础设施的关键原因。

主持人： 你当初肯定没预料到，TPU后来会成为Transformer架构的基础——毕竟Transformer架构比TPU出现得晚得多。

Jeff Dean： 对，这也是为什么我们造的是一个通用的线性代数系统，也就是TPU真正的本质。因为我们知道机器学习算法还在不断演变，你不想过度专用化，但又要专用到足以获得显著的性能提升——比如可以有非常大的乘法器阵列、高速内存、高速互连，后来的TPU还能让许多芯片高效地协同处理同一个问题。这么多代下来，我们一直在持续扩展和提升它们的性能。

主持人： 这餐巾纸算账真是厉害。那对于在座想成为未来创业者的人来说，今晚回去应该做哪种餐巾纸测算，才有可能做出像TPU这样有分量的东西？

Jeff Dean： 这个不好一概而论。我觉得应该去想想，你关注的领域里有哪些问题，有哪些瓶颈，是否存在截然不同的解决思路，能带来一到两个数量级的性能或能力提升。有时候，如果你眯着眼睛看一个问题，不去纠结这个问题今天是怎么解决的，而是从第一性原理出发去思考该怎么解决，你就可能想出一些真正好的点子——很可能是别人还没想到的。

主持人： 是个不错的建议。多年前，你写过一份很出名的清单，叫“每个工程师都应该知道的延迟数字”，里面列了缓存未命中要多久、磁盘寻道要多久、一个网络包从加州传到荷兰要多久，诸如此类关于分布式系统和系统工程的数字。这份清单后来被很多分布式系统工程师奉为圭臬。现在这份清单也该更新了，能不能给我们一个2026年的AI版本？

Jeff Dean： 如果放在今天的AI系统里看什么是重要的，你会想知道加速器主存到片上内存、再到乘法器单元之间的带宽是多少；做一次乘法运算需要消耗多少能量；芯片之间的互连带宽是多少，这个带宽能连接多少块芯片；再往外扩展，比如要和一万块芯片通信而不是五百块时，网络带宽会怎样下降。我觉得这些都是非常重要、值得了解的数字，会实实在在地影响你思考和解决具体问题的方式。

主持人： 你说过一个很有意思的观点，现在衡量一切的单位是能量。你提到做一次运算大约消耗1皮焦耳，但搬运数据、做数据I/O要消耗上千倍的能量。

Jeff Dean： 对，光是把数据从加速器的HBM取到处理器里以便计算，就是这个量级的开销。

主持人： 这个差距其实悄悄决定了什么产品能做出来、AI算法该怎么设计。那么，创业者常常以为是“模型问题”，但其实是能量或数据I/O问题的，都有哪些？

Jeff Dean： 你提到的搬运数据和实际计算之间上千倍的能量差距是个很关键的例子，它塑造了我们做机器学习时的很多方面。如果没有这上千倍的差距，就不需要做批处理（batching）；但正因为有，你必须一次性处理很多样本或很多token，才能把这次数据搬运的开销摊薄，这样才



不用承受上千倍的减速，而是承受"上千倍除以批大小"的能量开销。但对于真正的低延迟场景，批处理效果并不好。这类围绕计算机硬件能量消耗的决策，实际上深深影响着我们构建上层系统的许多选择。

主持人： 一个很具体的例子就是模型训练本身——把数据集分批、跑epoch这套流程，大家可能以为这是"模型问题"，但其实是系统层面的数据I/O问题，对吧？

Jeff Dean： 对，你必须把数据组装成批次才能提高硬件效率。理想情况下你可能想做批大小为1的训练，但这样效率不够高，所以现在大家用的批次都相当大。

主持人： 你以擅长利用一个长周末，独自钻研出一个精妙方案而闻名。有没有可能Jeff你花上几周时间，去实现批大小为1的训练？

Jeff Dean： 我最近其实在更多地思考推理。推理是个挺有意思的问题，因为你确实需要非常低的延迟——训练则未必需要那么低的延迟。我觉得针对推理去做更专用的硬件，还有很大的空间，目前这方面做得还不够。

主持人： 在推理方面，有哪些你正在深入思考的有意思的事情？

Jeff Dean： 主要是尽量减少数据搬运；尝试极低精度的运算，可能不需要支持那么多精度——如果你已经明确知道自己需要什么精度，那就直接把它内建到硬件里，别的都不要支持太多。

主持人： 这让我想起一位知名计算机科学家的类比：AI本质上是一个巨大的压缩问题——因为要把数据做有损压缩再还原，你首先得真正理解这些数据。

Jeff Dean： 对，如果你真正理解数据，就应该能把它压缩得很好，而Transformer架构正是目前证明行之有效的方法之一。

主持人： 效果确实不错——这也是我同事们的功劳。现在我们把视野放宽一点。过去AI的进步主要是"模型变得更好"——更多数据、更大参数量。但最近这一两年，进步越来越体现在模型之外的东西上：检索、工具、记忆、智能体工具，这些可能正在汇聚成大家所说的"上下文工程"（context engineering），对吧？

Jeff Dean： 对，模型只是你要构建的整体系统中的一部分，这个系统要能解决非常复杂的问题——这涉及模型如何使用各种工具，如何检索相关信息，可能还要保留过去解决其他问题时检索到的信息，并把这些信息放进模型的上下文中。这样做的好处是，这些信息对模型来说是非常清晰的，不像训练数据那样——训练数据是数万亿个token混合搅拌进数千亿甚至数万亿参数里，相对模糊；而当下这个具体问题或用例的上下文，模型能直接、清晰地看到。接下来，模型需要理解有哪些工具可用，哪些工具能帮它解决下一步的问题，如何把一个问题拆解成一连串的工具调用，也许还要尝试多种方案并评估哪种有效——这整套复杂智能体乃至多智能体系统的编排，我认为会变得越来越重要，也是一个非常令人兴奋的领域。

而且我觉得这个问题领域有意思的地方在于，在座的每个人其实都能参与其中——因为以前训练一个模型需要海量的资源、GPU和数据，但做上下文工程，你只需要一个类似Gemini的API接口，然后自己搭建检索系统、工具调用之类的东西就行了。

主持人： 那大家应该怎么做，才能在上下文工程上做得更出色？

Jeff Dean： 一个很好的办法是，实际用这些模型、脚手架（harness）和工具去解决问题，有时你能亲眼看到模型在哪些地方失败了。往往你不需要去调整模型参数（这从外部很难做到），而是可以通过给模型制定更好的指导规范、写"技能"（skill）让模型知道如何使用不同的工具，



就能显著提升模型解决某一类问题的表现。做这件事的过程中，你所使用的整套系统会不断自我改进。这也是一个很好的方式，能让你更清楚模型还需要哪些额外信息才能变得更强大。

主持人：能举个你自己亲手做的上下文工程的例子吗？比如你写过的某个技能或工具，对你的工作流程带来了很大改变？

Jeff Dean：几周前，我和Sanjay在做一些底层库的性能优化工作。我们在Google写了一个微基准测试库，可以测量各种操作要花多长时间，或者填充某个数据结构要多久——这些数据结构可能在Google的数百万个进程中被使用，所以确保它们高性能非常重要。你可以写微基准测试，但如果没有智能体系统，通常的流程是：先测量当前基准的性能，做一些你希望能提升性能的修改，然后重新运行基准测试看哪里有改善，再运行更大范围的基准测试，测量缓存占用情况等。于是我们写了一个“技能”，教模型如何按顺序完成这一整套流程，让它能自主完成“测量基准—改进代码—测量性能提升—迭代”这个循环，这对某些类型的问题效果相当不错。这其实就是把我们自己作为工程师会用的方法，以模型能使用的形式教给它。

主持人：这听起来很厉害。你是说，如果有人能拿到这个技能，就能像Jeff Dean一样做性能优化。这好像是价值连城的东西啊。

Jeff Dean：我们几个月前其实发布过一份文档，叫《性能提示》(Performance Hints)，是我和Sanjay写的，大概30页，讲各种性能优化技巧。已经有人把这份文档摘要后喂给不同的模型，模型在推理代码性能问题时确实变得更大了。

主持人：大家都听到了，只要拿这份公开的《性能提示》文档，你也能像Jeff Dean一样优化自己的代码，而且完全免费，大家都该去试试。现在说说智能体的话题——在座应该都在做或做过智能体，我相信大家都见过自己的智能体在第30步或第40步的时候“脱轨”。智能体在前十步左右表现不错，到五十步左右就开始摇摇欲坠了。你觉得今天的瓶颈是什么？是上下文、评估器的问题，还是因为它本质上是开环系统，误差会累积？

Jeff Dean：我们当然希望智能体能长时间稳定运行，因为这样才能解决更复杂的问题。但正如你观察到的，它们有时候在跟工具交互了十来次之后就会开始“失灵”。这有时是因为模型在做一些它经验不足的事情——它是在某个整体数据分布上训练出来的，一旦稍微偏离它熟悉的分布，就像大多数机器学习模型一样，表现会突然下降，偏离得越远，效果越差。

所以有几种应对方式：一是给模型提供“技能”和提示，让它尽量停留在它熟悉、把握较大的路径上；二是采用多智能体系统，让多个智能体尝试不同方案，再用另一个模型或智能体去评估哪些方案更有希望——这本质上是在搜索可能的解空间，保留看起来靠谱的方案，丢弃偏离轨道、效果不好的方案。这是一个非常通用且有效的技巧：用推理时计算（inference-time compute）去搜索可行的解题路径，能大幅提升长程智能体流程的性能和可靠性。

主持人：你们内部是怎么实现这种流程的？

Jeff Dean：我们有各种脚手架和整套技能，尤其是在Google内部的开发环境里。我们准备了一系列技能，让智能体知道如何使用我们内部的各种工具——编程、代码审查、性能测量、抓取日志文件等等。这些技能能让基础模型的能力大大增强，尽管它本来并没有专门针对Google内部工程师如何从我们专有系统里抓取日志文件这类操作做过训练，但只要有了合适的技能定义，它就能真正学会使用，从而提升智能体的实用性。

主持人：现在我们聊聊创业公司能在哪些地方胜出。这一部分我个人非常关心，因为在座的每个人都要决定未来要做什么、要成为怎样的创业者。Google的特点是，从处理器到产品，整条链路都是自己协同设计的。哪些层面会是像Google这样的公司持续深耕、不断做强的？两三个人的团队又能在哪些地方胜出？

Jeff Dean： Google和我们的Gemini模型、硬件基础设施，都在努力打造能做几乎任何事的通用模型。但这也意味着，我们在很多具体领域上的关注是有限的。而如果有人能针对某个特定领域，做出一个设计精良的界面，配上一个专用模型或一套技能——这个模型不属于我们通用模型体系里的那种"大杂烩"——就有可能做出体验上乘、准确率很高的产品，特别是在你真正热爱、深耕的领域。我觉得这正是两三个人在一个房间里、专注做他们真正热爱的事情所拥有的优势。但我也要提醒一句：通用模型确实在越来越多的领域上变得更强了。所以你得想清楚，你正在做的事情是不是够"持久"——未来六个月或十二个月，前沿模型会不会就能做到这件事了？还是说，这件事可能还要两三年模型才能做到？这是你在决定做什么之前应该权衡的问题。

主持人： 那我们再深入一点。通用模型你们肯定会持续做下去、不断变强，那创业者应该怎么去判断哪些领域是值得投入的？

Jeff Dean： 我觉得最重要的一点是，选一个你真正兴奋、想去做，并且相信对世界有用的问题。如果你做到了这一点，你就已经领先一大截了——相比那种一觉醒来就觉得"我其实不太想干这个"，或者做的东西对世界、对大多数人其实没什么用的情况。这是我为自己选择下一个要研究的问题时应用的首要标准。

其次，你要看现有的通用模型在这个问题领域能做到什么程度——可以去测试，看它们能不能很好地完成这件事。如果它们完全做不到，那通常是个好迹象；如果它们能做一部分，但做得不太好，那反而不一定是好事，因为这往往说明这项能力正在这些模型里萌芽，随着训练数据增多或模型规模扩大，它们很可能会很快变强。所以你应该去找那些模型成功率是0%或1%的问题，而不是20%。

主持人： 那怎么找到这样的问题？这些问题是不是本质上就在训练数据的分布之外？这类问题的形态是什么样的？

Jeff Dean： 有时候，是你要构建的产品能接触到某种特殊数据，而通用模型接触不到。比如你在做一个帮用户整理个人信息的产品，通用模型本身没有这些数据的访问权限，这时你的产品就有很大优势，因为你突然拥有了对重要数据的可见性。

也有可能是某个极其困难的问题，如果你拿到合适的训练数据，训练一个比通用模型更专用的模型，成本反而可以做得很低——训练一个针对特定问题的小众模型，也许不需要太多算力，却能得到一个精度很高的模型。这有时候是解决某个通用模型处理得不太好的重要问题的绝佳切入点。

主持人： 我觉得这里其实有两条路径：第一条路有点滑稽，因为你们已经在"整理全世界的信息"了。

Jeff Dean： 对，那块基本上被覆盖得很充分了。

主持人： 但"整理你个人的信息"却还是空白，这很有意思。第二条路径，你提到了在某些特定领域做更专用的模型，能不能再展开讲讲这些领域有哪些？

Jeff Dean： 比如我同事做的AlphaFold，就是一个非常专用的蛋白质折叠模型，非常成功，很好地处理了那个特定领域，让你一下子拥有了能回答蛋白质结构相关问题的强大工具和模型——但它不是一个通用模型，而是一个非常专用的模型。还有其他一些领域也适合这种做法，比如材料科学、芯片设计等，都能让你利用一个精度很高但小众的模型，去做那些今天还很困难的事情。

主持人： 这是个很好的例子。所以如果在座有人能找到和AlphaFold形状类似的问题，那可能就是值得做的方向。假设大家已经找到了要做的问题，接下来聊聊怎么成为一名真正优秀的"AI原生"创业者。你以前说过，管理一支50到100个智能体组成的"舰队"，关键在于写出非常清晰精准的设计文档和规格说明（spec）。人们要怎样才能擅长这件事？这些文档具体长什么样？



Jeff Dean： 如果你能清楚地说明自己想要什么，跟虚拟智能体协作时就会顺利得多。你把想要的东西说得越清楚，智能体就越有明确的准则和目标框架可以遵循；反过来，如果你说得含糊，智能体就得自己去推测你的意思，而很多情况下它推测出来的会和你脑子里想的不一样。从计算机科学诞生之初，我们就一直告诉大家：写代码之前，先把软件要实现的目标讲清楚，这非常重要。现在我们有能真正写代码的智能体系统，但把想要的东西讲清楚这件事的重要性反而更高了——因为以前你是把任务交给一个非常聪明的人，他有背景知识，也可以反问你；智能体有时候也能做到，但我认为清晰的规格说明依然极其重要。

举个编程智能体表现特别好的例子：现在的模型在把软件从一种编程语言翻译成另一种语言时非常高效，因为这种情况下你其实拥有一份极其详尽的规格说明——整套软件本身就说明了系统该做什么。所以如果你有一份Python实现，想要一份Go实现，现在的模型在这方面能力相当强：它可以拿Python里所有的测试用例，确保在Go版本里也能通过，把测试翻译成Go，比较两个实现之间的行为差异，直到没有差异为止——效果非常好，因为规格说明本身就非常清晰。

主持人： 假设未来每个创业者都能熟练地同时运行成百上千个智能体，所有代码都由智能体写出来了，那时候什么会成为稀缺的技能？

Jeff Dean： 我认为是要有非常好的判断力（taste），知道该让智能体去做什么。这其实是做研究的核心问题——一个研究者可能拥有所有的工具和技巧，但真正决定成败的往往是：你选择把时间花在什么问题上。如果你选对了问题，并且成功解决了它，这远胜过你以很漂亮的方式去执行一个相当平庸的研究课题。所以那种“该做什么”的高层判断力，我认为极其重要。而模型未必擅长做这种判断，所以未来会是人来引导大量的AI辅助计算，从而更快地实现伟大的成果——但那个“你想要模型做什么”的核心，才是你应该专注的关键。

主持人： 那我们再深入聊聊“判断力”，因为在当前这个智能体编程的时代，这个词被谈论得很多。你具体是怎么培养判断力的？听起来有点玄乎，怎么把它落到实处？

Jeff Dean： 这确实不容易，判断力在很多情况下并没有一个可衡量的客观标准。我觉得一部分来自经验——过去做过很多不同的问题，会让你对未来什么问题可能有意思、什么东西也许只是把过去的方法拼凑一下就能勉强做出来、还有哪些开放问题需要你去攻克才能做出真正神奇或极其有用的东西，形成一种直觉。

另一个积累经验的方式是：写下一份你认为未来12个月可能重要的事情清单，也许你只挑其中一件去做，但十二个月后回头看看，评估这些事情里哪些真的变得重要了，哪些世界上其他人已经去做了、哪些还没有人做。这能给你的判断力积累更多的样本。这是一项很重要的技能。

主持人： 我记得我们之前聊到，第三种方式是做一些非常“疯狂”的思想实验。

Jeff Dean： 对，这也是一个好办法。我觉得有时候，不要把大多数人视为理所当然的事情当成理所当然，是件好事。前几天我和几个同事就做了一个疯狂的思想实验：60年来，整个半导体芯片设计和制造行业一直在努力把晶体管做得越来越小、错误率越来越低——因为我们默认，同一设计生产出来的每一块芯片都应该和其他每一块完全一致，不能有任何比特翻转。所有内存都有ECC纠错机制之类的各种误差余量设计。但在我们构建大规模分布式系统的宏观层面，我们并不做这种假设——比如我们用不可靠的零部件构建可靠的大规模分布式文件系统：单个磁盘可能会坏，但你的数据应该是安全的，因为我们在更高层面有机制来保证这一点，比如把同一份数据放在三台不同机架上的三台不同机器里，这样即使某个机架交换机、某台机器或某块磁盘出故障，数据依然安全；我们还有里德-所罗门纠错编码技术。但在晶体管级别的技术尺度上，我们似乎并没有把这套思路做到极致。

于是一个有意思的思想实验就是：如果尝试用一种每天可能出错20次的晶体管去构建系统，而不是要求百万年才出错一次，会怎么样？那会是完全不同的设计出发点，可能会让你在芯片制造端做出一些非常有意思的东西——你会有截然不同的设计方法，因为如果要把信号从一个地方传到另一个地方，而你用的是这种极不可靠的晶体管，你可能需要非常不同的信号传输方式，比如



沿着多条冗余路径发送信号，确保至少有一条能成功送达。我认为这会是一组相当有意思的思想实验。我不是说我们真的应该去做这件事，但偶尔质疑一下这些默认假设是有点好处的。很多时候，这些思想实验并不会真正落地，因为过去50年我们选择某种做法而不是另一种做法，往往是有充分理由的。但时不时地重新审视这些假设，还是很有价值的。

主持人： 这太狂野了。这听起来和神经形态计算，或者说人脑的工作方式很像。

Jeff Dean： 对，确实如此——大脑中信号从一个地方传到另一个地方的可靠性也不算特别高。所以我认为，大脑里那些真正重要、需要从一处传到另一处的信息，会有多条通路来实现这种冗余传输。

主持人： 那么，在你职业生涯里，有没有哪个你曾经打破的“疯狂假设”，最终真正构建出了有分量的系统？

Jeff Dean： 有几个后来确实成功了。TPU就是个好例子——在那个问题领域被认为重要之前，就为一个非常小众的问题专门做硬件优化，这就是一次思想实验。

MapReduce的起源也是个好例子。当时我、Sanjay，还有几位同事一直在做Google网页爬取和索引系统的各个迭代版本，我们写了大量手工并行化的代码，加上很多检查点机制，来确保系统在跑在一百台、一千台机器上、而其中一些机器出故障时依然稳健可靠。但这些代码常常和你真正想做的相对简单的事情——比如“我只是想看看所有网页的内容，然后顺便算出URL到语言、文本内容的映射”——混杂在一起，被大量的并行化和可靠性代码淹没。于是我们想起了自己在函数式编程语言方面的训练，意识到可以眯着眼睛看这些问题，开发出MapReduce这样一个抽象层，放在具体实现之上；而具体实现层下面则可以把所有的检查点和可靠性机制都封装进一个底层库，供上层所有代码复用。这后来成了Google处理超大规模计算任务的一种极其成功的稳健可靠方式——正是从“如果我们眯着眼睛看，能不能找到很多适配这个抽象的问题”这个思想实验开始的。

主持人： 了不起，这个思想实验最终催生了MapReduce。回到你刚才提到的、你目前正专注研究的定制硬件话题上——现在有AlphaChip负责芯片布局，还有AlphaEvolve能提出方案、评估方案，并保留有效的方案。听起来你们正在打造一整套能够自我迭代、“用AI打造AI”的系统。

Jeff Dean： 更广泛地说，这本质上是科学方法的根基：提出一个实验，实现运行这个实验所需要的东西，评估实验结果，从中得到结论。现在有越来越多的问题，已经可以实现“不只跑几个实验，而是跑成千上万个实验”，因为你能把这整个循环自动化，把这个循环的延迟降到极低，这将非常重要，能让我们在科学、工程、机器学习模型设计本身，乃至芯片设计等工程任务中取得突破。

如果你能以自动化的方式做这些事情，并且有一个能把高层目标拆解成子问题的编排框架，每个子问题都可以用这种自动化循环去探索最佳解法，再有一个编排框架把各个子问题的解法整合成针对高层问题的整体方案——这会带来巨大的影响，能加速机器学习本身的进步，也能加速科学和工程的进步，我觉得这会非常了不起。

主持人： 听起来很多领域，只要能有很好的评估器，尤其是那些能被形式化验证的领域，都很适合这种AI自我迭代的系统。

Jeff Dean： 对，不过很多情况下，你的评估器本身需要做得更快。举个例子，我的同事大概十年前在量子化学的一些问题上做过一项工作——要理解某个分子的性质，你可以生成一个分子构型，然后想知道它有哪些性质。你可以运行一个计算量非常大的密度泛函理论（DFT）模拟器，可能要跑一整夜才能给出一个分子的答案。但我的同事们用大量这类模拟运行的输入分子构型和输出结果，训练出了一个模拟器的神经网络近似模型——这仍是个验证工具，但原本要跑一整夜的计算，现在快了30万倍，而且准确度几乎和跑完整模拟器一样。这就彻底改变了做科学研究的方式，因为原本要花六个月东拼西凑算力才能跑完的一千万种情况的筛选，现在你吃个午饭的



功夫就能做完。我认为在很多领域，都有很大空间去做速度快得多的验证模型，也许是可学习的验证模型，能在极短时间内给出接近真实答案的近似结果，这会改变这些实验循环的运转方式，以及你能多快地跑完这些循环。

主持人：你最期待，这种被大幅加速的科学方法，能在哪些领域和问题上取得突破？

Jeff Dean：机器学习本身显然是一个——能不能有一个模型，通过运行大量实验来递归地自我改进？如果你想今天大型研究团队是怎么改进模型的：人们想出一些点子，跑一批小规模实验，看哪些效果好，把效果最好的挑出来，在更大规模上尝试，再评估，最后把结果整合成新的模型配方。我认为没有理由不能让这个循环变得更自动化——模型自己决定去探索，或者有人在最高层稍微推一把，比如“你可以尝试一些结合了这个思路的新模型架构”，然后模型会去跑大量实验，看哪些有效，再以快得多的速度把结果整合进去。本质上，你是在优化“每单位算力投入所能获得的发现数量”。

主持人：非常有趣。最后一个问题：在座的各位未来都会成为创业者，或者开启自己的职业生涯，你们会收到很多拒绝，这是必然会发生的事，Jeff你自己也经历过。有个故事是，2014年你和Jeff Hinton、Oriol Vinyals一起写了一篇关于“蒸馏”（distillation）的论文——用一个大的教师模型来训练一个更小、更高效的学生模型，参数量更少，计算成本更低，这个技巧现在已经被整个行业广泛使用。而这篇论文当时在NeurIPS被拒了。

Jeff Dean：对。我不怪那个项目委员会，因为很多时候一篇论文只有三份评审意见，其中一位评审看了之后说，这篇论文“不太可能有重大影响”。但我们当时写这篇论文的时候，其实认为这是一个非常重要的问题，因为我们迫切希望能从大规模模型中做出更便宜、同样强大的模型，好把模型服务推广给语音、视觉等各个领域的更多用户。不过评审可能没有那样的经验，也许他们没有从大规模AI服务的角度去思考，而是在想这是不是一项“根本性”的进展。所以论文时不时会被拒，这很正常，我们就把它放到了arXiv上，大家读了、用了，一切都好。而且我们现在做Gemini里的Flash系列小模型时，其实就用到了这个方法——从我们更大规模的Pro模型蒸馏而来，这也是为什么Flash模型相对于它们的体量和速度而言表现如此出色的部分原因，在同尺寸模型的基准测试里也是数一数二的。

主持人：确实令人印象深刻。我想这里面的一个教训是：即便被拒绝了，也要继续前进。

Jeff Dean：对，这就是我从中总结出来的教训。

主持人：有意思的是，你1999年加入Google的时候，那还只是一家20人的创业公司。如果把当年那个年轻的Jeff Dean，用他当时的技能，穿越到今天这个时代，你会怎么做？加入前沿实验室，还是自己创业？我不知道，你会做什么？就是“今天的25岁Jeff主题”。

Jeff Dean：这个真的很难说，也是一个非常个人化的选择，取决于你想把时间花在什么上面。对我来说，最重要的问题是：你要做的是不是你真正在乎的事情？如果你和一群你喜欢共事的同事一起，能在这件事上取得进展、集体把它解决或推进，它会不会以某种积极的方式，给这个世界带来改变？比如说，你能不能突然做出一样东西，帮到生物化学家，或者更广泛一点，帮到程序员、帮到所有互联网用户？你应该努力做的，是对世界产生积极影响，和你喜欢共事的人一起工作，并且努力做到最好。

至于你提到的那个具体权衡——加入前沿实验室，还是和一两个、两三个亲近的朋友一起创业，我觉得这是两种不同的体验。在一个大型、成熟的组织里，你会有一定的架构，有很多很多了不起的同事，他们知道很多你不知道的东西；你会有很多有意思的问题可以研究，而且你已经有一个现成的平台，可以通过你的工作直接影响很多人。而作为一个非常小的创业团队，你必须对自己在做的事情充满热情，而且要承担很大的风险——能不能成功，能不能把这件事发展壮大，都是未知数。但这个过程也可能非常有成就感。所以我觉得这真的是个人偏好的问题，但无论走哪条路，至少你应该问自己：如果我去做这个问题，并且出现了最好的结果，这个世界会不会因此



变得更好？还是说，世界只会觉得“这挺酷的，但也就那样”？如果是后者，那可能就不值得你花时间去做了。

主持人：那我们再多聊聊第二条路——和真正喜欢的人一起，在一个小团队里工作。你一直是很多工程师心目中出色的导师和管理者，也主导构建了很多庞大的系统。对于怎样与聪明人共事、怎样找到聪明人，你有什么经验可以分享？

Jeff Dean：你总是想找到那些在团队所需要的某个领域拥有真正过硬技能的人——不管是在公司内部组建团队，还是自己创业。但你同样也想找那些和你相处起来很愉快的人，因为你们会花大量时间一起攻克非常困难的问题，你需要的是那种低自我（low ego）、有团队精神、和你技能互补的伙伴。我一直觉得，在一个小团队里工作特别有意思——团队里有人知道我不知道的东西，我也可能有别人不太具备的技能，大家一起构建、攻克某个人无法独立完成的东西。而在这个过程中，你其实获得了大量新的知识和技能，其他人也是如此。你应该把自己的工程或研究生涯，想象成拥有一条了不起的“工具腰带”，你要不断往上面添加新工具——因为你永远不知道什么时候会遇到一个需要四种专门工具而不是三种的问题。工具越多，你未来能解决的问题也就越多。

主持人：最后一个问题。我相信在座的某个人，或者好几个人，未来一定会做出像 MapReduce、TPU、蒸馏这样有分量的东西。你希望他们会去做什么问题？

Jeff Dean：世界上有意思的问题太多了，我随便列举几个，肯定不是全部，因为这个世界太大了，问题也太多。我个人特别期待硬件方面的新思路——就像我们刚才那个思想实验，或者说更高效的推理硬件。我认为机器学习也许存在截然不同、比我们今天使用的方法数据效率高得多的算法。想想我们今天的大规模模型，它们看到的数据量大概是一个18岁人类的一千倍，但一个18岁的人在很多方面依然比这些见过更多数据的前沿模型表现更好。那么，能不能设计出数据效率高得多、能持续从自身行动中学习的系统？持续学习（continual learning）是个非常有意思的方向。我觉得多智能体交互也很有意思。还有，能不能创造出让人与人之间进行更好交流的方式？有没有办法让对话更加文明、更有建设性？有没有办法基于兴趣帮人们认识全世界他们本该认识的人？这些都是挺有意思的问题。这个世界上还有很多很酷的事情，我们都应该努力去创造更酷的东西。

主持人：太棒了，非常感谢你，Jeff Dean。今天就到这里，谢谢大家。

Jeff Dean：谢谢大家。

风险提示及免责声明

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

* 体验资格每人限申领一次

扫码

免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

269 收藏

分享到:

写评论

请发表您的评论





表情



图片

发布评论

